

cudaHostAllocMapped

>

DSCH**Oct 14 '09**

Hey!

Can anybody tell me what is the differenz between

cudaHostAlloc (void **ptr, size_t size, cudaHostAllocMapped)

and

cudaHostAlloc (void **ptr, size_t size, cudaHostAllocDefault)?

I don't understand the advantage of mapped page lock memory in opposite to normal page lock memory. In both cases the device can access the host memory direct. Which advantage has an address mapping? And which methode is faster?

best regards!

MisterAnderson42**Oct 14 '09**

Mapped memory can be accessed directly from within kernel **without** needing to cudaMemcpy the region back and forth. You need to get a different pointer to do this on the device, though, through a cuda API call that I don't remember the name of at the moment.

DSCH**Oct 14 '09**

Mapped memory can be accessed directly from within kernel **without** needing to cudaMemcpy the region back and forth. You need to get a different pointer to do this on the device, though, through a cuda API call that I don't remember the name of at the moment.

Thanks for reply!

Ok, so cudaHostAlloc (... , cudaHostAllocDefault) works in the same way as cudaMalloc(...), with the differenz that cudaHostAlloc (... , cudaHostAllocDefault) uses no paging. Right? But why is using page locked memory faster?

YDD**Oct 14 '09**

Page-locked memory is faster because the GPU can only DMA to page-locked memory (if it's pageable, then the pages might get swapped out mid-transfer causing Bad Things to happen). This means that cudaMemcpy internally does a copy to a page-locked buffer, and then has the card DMA to that. This extra host-side copy slows things down. If you use cudaMallocHost or cudaHostAlloc, then the driver knows that the buffer is pinned and can skip the copy to the internal buffer.

DSCH

Oct 15 '09

Page-locked memory is faster because the GPU can only DMA to page-locked memory (if it's pageable, then the pages might get swapped out mid-transfer causing Bad Things to happen). This means that `cudaMemcpy` internally does a copy to a page-locked buffer, and then has the card DMA to that. This extra host-side copy slows things down. If you use `cudaMallocHost` or `cudaHostAlloc`, then the driver knows that the buffer is pinned and can skip the copy to the internal buffer.

Thanks for reply!

Ah ok! But that brings me to a other questions:

1. Why do I have to use `cudaMallocHost`, `cudaHostAlloc` for asynchronous data transfers, if `cudaMemcpy`, `cudaMalloc` use page-locked-memory (by the internal page-locked buffer), too?
2. The page-locked buffer is an area in the working space?

regards!

YDD

Oct 15 '09

To answer question 1, it's probably a matter of complexity and expectation. People who want to do asynchronous transfers are obviously after high performance, and hence are probably already using page-locked RAM. With that existing effort, why complicate the driver with managing multiple internal buffers? For question 2... what do you mean by 'in the working space?' Page-locked memory is accessed just like normal memory. It's just that the OS knows not to swap those pages out.

New & Unread Topics

Topic	Replies	Views	Activity
Async thrust operation launches appear serially processed in nsight systems cuda performance parallel-computing	4	8	3h
☒ [q] mask in __shfl_sync() cuda kernel	3	9	3h
Potential memory leak - compute-sanitizer shows nothing camera cuda jetson	6	36	4h
Is there anyway to query the number of remaining operations in a stream?	2	5	4h

Topic	Replies	Views	Activity
How to seperate declare header and implement files in cuda programming? cuda	6	16	4h
What is a transaction from HBM to L2?	2	7	5h
How to use nvrtc && nvjit? cuda	1	7	7h
What's the difference between CUDA stack and local memory?	1	8	9h
It seems that CUDA kernel are not running parallel	5	11	11h
How to use the `ex2.approx.f16x2` instruction?	2	27	Aug 29

There are [191 new](#) topics remaining, or browse other topics in